Detecting Phishing Websites with Machine Learning:
a reproduction and critique

Final Project – Data Science in Cyber (Dr. Uri Itai)

Guy Segal · June 2026

Source critiqued: *"Detecting Phishing Websites using Machine Learning"* by Sayak Paul, published under Intel Software Innovators on Medium (medium.com/intel-software-innovators/detecting-phishing-websites-using-machine-learning-de723bf2f946), with the companion GitHub repo github.com/sayakpaul/Phishing-Websites-Detection.

1. Summary of the Source

The article picks a pretty well-defined problem: given a website, decide whether it's phishing or legitimate, using a fixed table of pre-computed features rather than the raw page itself. That's a sensible framing for a tutorial – it sidesteps the (much harder) problem of scraping live pages and extracting features in real time, and just focuses on "given these 30 numbers, what's the label." The author is upfront that phishing is a real and costly problem (stolen credentials, financial fraud, etc.) and that catching it automatically, rather than relying purely on blacklists, is the motivation.

Why it matters: blacklists are reactive by nature, a site has to be reported and verified before it lands on the list, and by then plenty of victims have already clicked through. A model trained on structural features of a URL/page can in theory catch a brand-new phishing site on day one, which is the whole appeal of the machine-learning angle here.

Dataset

The **UCI Phishing Websites dataset** – 11,055 rows, 30 features, all pre-encoded as small categorical integers (mostly -1/1, a few -1/0/1), target column Result (-1 = phishing, 1 = legitimate). Features fall into a few natural groups: things you can read straight off the URL (IP address used instead of a domain, URL length, presence of "@", use of a shortener, prefix/suffix tricks like "paypal-secure.com"), things that need a DNS/WHOIS lookup (domain age, registration length, DNS record), and things that need the actual page content (favicon source, form action / SFH, anchor tag destinations, iframes). It's a well-known dataset, originally published alongside a 2014 paper by Mohammad, Thabtah and McCluskey that explains how each feature was engineered, which is more documentation than most Kaggle datasets get.

Proposed solution / methodology

The author trains three models in increasing order of sophistication: a logistic regression baseline, a small Keras neural network, and finally a model built with **fastai** using entity embeddings for the categorical features, a 1cycle learning rate schedule, and discriminative learning rates. The -1 labels are remapped to 0 "to stabilize optimization." No feature scaling is applied (values are already small and bounded) and, notably, the author explicitly decides *against* doing any feature selection, reasoning that all 30 features contribute collectively. An 80/20 train/test split is used, with shuffling.

Model	Reported accuracy
Logistic Regression (baseline)	93.71%
Neural network (Keras, Adam optimizer)	96.52%
Fastai model w/ entity embeddings + 1cycle policy	97.02%

The author's headline claim is that the final fastai model "matches state-of-the-art" results reported elsewhere in the literature for this same dataset, and that the gain over the plain neural net comes from the entity embeddings doing a better job representing the categorical features than one-hot/ordinal encoding would.

2. Critical Evaluation

This is the section where I think the article runs into trouble, and it's worth saying up front: my issue isn't with the code or the explanations, which are clear and the repo runs fine. It's with two specific claims, and with a methodological detail the author (like most tutorials using this dataset, mine included on the first pass) never checks.

Claim 1: the entity-embedding model earns its complexity

The author frames the fastai model's 97.02% as the payoff for a noticeably more involved pipeline – entity embeddings for categorical variables, a 1cycle learning rate schedule, discriminative learning rates layer-by-layer. Embeddings are a genuinely good idea for *high-cardinality* categorical features (think zip codes, product IDs, thousands of categories) where one-hot encoding would blow up the dimensionality and a dense vector representation lets the model learn useful structure. But every feature in this dataset has only 2 or 3 levels. There's just not much "structure" for an embedding to discover in a 3-category variable that a one-hot encoding wouldn't already capture directly. In my own reproduction (section 5 below), a default **Random Forest with zero tuning** hits 97.7% accuracy on the same split methodology the author used, and 96.5% once I fix the data leakage issue described below – either way, in the same ballpark as the entity-embedding model, for a fraction of the engineering effort and no GPU. I don't think the article's data supports the implicit claim that embeddings are the right tool for this particular dataset; it supports "you can also get to ~97% with a much fancier model," which is a different and weaker claim.

Claim 2: "no feature selection needed because every feature contributes"

This one is asserted rather than tested in the article – there's no ablation, no feature-importance plot, nothing showing that removing a feature actually hurts performance. My own correlation and mutual-information analysis (section 3 of the notebook) tells a pretty different story: two features, SSLfinal_State and URL_of_Anchor, carry the vast majority of the signal (Spearman correlation with the target of -0.74 and -0.70 respectively, far ahead of anything else), and several other features are so lopsided (95%+ one value) that they can't be contributing much no matter what the model is. That doesn't mean feature selection would necessarily *improve* accuracy – tree models in particular are pretty good at ignoring useless features on their own – but "every feature contributes" as stated isn't backed by anything in the article, and a quick look at the data suggests it's at best an oversimplification.

The methodology problem the article doesn't check: duplicate rows

This is the part I'd flag as the most serious issue, and it applies to basically every tutorial built on this dataset, not just this one. **47% of the 11,055 rows are exact duplicates** of another row (identical features *and* identical label), and **71% of rows share their feature vector with at least one other row**. Once you do a random 80/20 split without deduplicating first, a big chunk of the test set ends up being rows the model has effectively already memorized: in my reproduction, **64.6% of the test rows have an exact feature-vector match already sitting in the training set**. That inflates every reported accuracy number to some degree, including the author's, including mine on the first pass. I re-ran the comparison after deduplicating (down to 5,785 unique rows) and Random Forest dropped from 97.7% to 96.5% – not catastrophic, but not nothing either, and it's exactly the kind of gap that should be checked and disclosed before calling a number "state of the art.

Is the evaluation methodology appropriate?

Mostly yes for what it's trying to show (a single train/test split is fine for a tutorial, nobody's claiming production-grade rigor), but it's missing two things I'd consider standard practice: cross-validation to get a sense of variance across splits, and a check for duplicate rows before splitting. I added both in my reproduction. The metrics chosen (accuracy + a classification report + confusion matrix) are reasonable but a bit thin for a security context; there's no discussion anywhere of the precision/recall trade-off or what a false negative costs versus a false positive, which feels like a missed opportunity given the domain.

Are the conclusions justified?

Partially. "You can get phishing detection above 90% accuracy with this dataset and these methods" is well supported. "The entity-embedding model represents a meaningful improvement and matches the state of the art" is the part I'd push back on – my reproduction suggests a much simpler model gets you almost all the way there, and the comparison wasn't done on a leak-free split to begin with. I'd call this an example of a generally solid tutorial overselling its most sophisticated step.

3. Feature Engineering Analysis

Short answer on whether feature engineering happened: not really, on the author's side. The dataset arrives pre-engineered (the heavy lifting, deciding which 30 properties of a URL/page are worth computing in the first place, was done by Mohammad et al. back in 2014, not by the article's author). The article's only real preprocessing step is remapping -1 labels to 0. I went further in my own notebook and actually tried a few things, with mixed results.

Encoding

The raw -1/0/1 integers implicitly treat the middle category as a number sitting between the other two, which is a real assumption for the eight 3-level columns (SSLfinal_State, URL_of_Anchor, having_Sub_Domain, etc.). I tried one-hot encoding those eight columns instead (30 features become 46) and re-ran logistic regression and Random Forest. Logistic regression improved a bit (92.8% $\rightarrow$ 93.9% accuracy, F1 0.917 $\rightarrow$ 0.930), which makes sense: a linear model benefits from not having to fit a single coefficient across three categories that might not actually be linearly ordered. Random Forest barely changed (97.7% $\rightarrow$ 97.2%, if anything very slightly worse, basically noise) – trees split on thresholds regardless of encoding, so there's nothing to gain there. This is a clean, evidence-backed example of "the right preprocessing choice depends on the downstream model," not something I'd have known without testing it.

Scaling

Not needed and I didn't apply it. Every feature is already bounded to [-1, 1]; I checked anyway (standardizing before logistic regression) and the accuracy moved by less than 0.1 percentage points. Would matter a lot more if any feature were a raw count instead of a pre-binned category.

Feature creation

I built a risk_flag_count feature: the number of nine "classic" suspicious-URL flags tripped at once (IP-in-URL, shortener, @ symbol, double-slash redirect, prefix-suffix dash, abnormal URL, mouseover tricks, pop-ups, iframes), on the security intuition that several weak signals together should be more convincing than one. It correlates with the target at only 0.10 Spearman, and adding it to Random Forest moved the F1 score from 0.9738 to 0.9718 – essentially no effect, within noise. The honest read: the individual flags I picked are each fairly weak on their own (none were in the top-10 by correlation or mutual information), so summing weak signals didn't manufacture a strong one. It's a useful negative result – a reasonable-sounding feature idea that the data just didn't support.

Feature selection

Mutual information and Spearman correlation rank the features almost identically: a strong top tier (SSLfinal_State, URL_of_Anchor), a moderate second tier (Prefix_Suffix, web_traffic, having_Sub_Domain), and a long tail of features each contributing very little in isolation. I also found three pairs of features with |Spearman| above 0.8 between themselves – Shortining_Service & double_slash_redirecting (0.84), Favicon & port (0.80), and Favicon & popUpWidnow (0.94, which has no obvious causal story and is probably an artifact of how the original dataset was assembled). Dropping the weaker half of each pair and re-running Random Forest changed the result by less than 0.3 points – consistent with the redundancy being real but mostly harmless for a tree-based model.

Dimensionality reduction

I ran a 2-component PCA purely as a visualization, not as model input (with only 30 already low-dimensional features there's no real need to reduce anything). The two classes separate reasonably along PC1, but PCA assumes roughly linear/continuous structure which these categorical features don't really have, so I'd treat this plot as a rough sanity check rather than a real analytical tool here.

Is there redundancy, and how would I tackle it?

Yes, in two senses: the three highly-correlated feature pairs above, and the dataset-level redundancy of 71% of rows sharing a feature vector with another row. I'd spot the first kind with a pairwise correlation scan (what I did) and the second kind with .duplicated() on the feature columns before doing anything else with the data. Tackling it: drop one feature from each correlated pair (cheap, low risk given trees barely notice), and deduplicate rows before any train/test split, full stop – that one isn't really optional once you know it's there.

Was the feature engineering meaningful?

The original 30 features themselves are genuinely well thought out – each one maps to a real attacker behavior (registering short-lived domains, hiding the real link behind anchor text, abusing favicon/iframe loading to mimic a legitimate brand) and there's solid mathematical intuition behind most of them (e.g. SSLfinal_State works because a valid, long-lived HTTPS certificate from a real CA used to be expensive/slow enough that phishing operations couldn't easily fake it – less true today, see the error analysis). What's less convincing is the implicit claim that all 30 are pulling their weight equally; two features dominate, and that's worth knowing before scaling this approach to a new dataset.

Additional features that could help

- A real temporal feature – certificate issue date / domain creation date as a continuous age in days rather than a binarized flag, which would let a model learn a threshold rather than relying on whatever cutoff the original feature used.
- WHOIS registrar reputation (some registrars are disproportionately used for throwaway phishing domains) – not present here at all.
- Visual similarity to known brands (logo/layout similarity scores), which catches the cases where the URL itself looks fine but the page is a near-pixel copy of a real login page – the kind of thing pure URL/structural features will always miss.
- Lexical/NLP features on the URL itself (n-gram entropy, edit distance to popular domain names) to catch typosquatting that the current feature set doesn't directly encode.

4. Reproducibility Analysis

On the whole, this article reproduces well, better than most ML tutorials I've tried to redo for past coursework. The dataset is a standard, citable UCI dataset rather than something the author scraped themselves and never published, which already puts it ahead of a lot of blog-post tutorials.

- **Code execution:** the GitHub repo's notebooks ran without modification for the data loading and the scikit-learn/Keras parts. The one piece I genuinely could not reproduce was the fastai entity-embedding model – fastai wasn't something I could get installed cleanly in this environment, and fastai's API has changed enough across versions that an old fastai v1 notebook is not a drop-in run today. I flagged this rather than fake it; my closest honest substitute is the small sklearn MLP in section 4 of the notebook, not a true entity-embedding model.
- **Files and dependencies:** the dataset is openly hosted on the UCI repository and downloads cleanly. Standard packages only (pandas, scikit-learn, matplotlib/seaborn) for everything except the fastai step.
- **Hidden preprocessing:** the only preprocessing the article itself does is the -1 $\rightarrow$ 0 label remap, which is stated explicitly, nothing hidden there. What *is* hidden, in the sense that it's not discussed anywhere in the article or in the original dataset documentation I could find, is the duplicate-row situation described in section 2. That's not something the author did wrong, exactly, it's a property of the dataset they didn't go looking for, but it is a hidden issue that affects how the reported numbers should be read.
- **Overall reproducibility:** good for the baseline and neural-net models, partial for the final fastai model due to tooling drift rather than anything the author did poorly. I'd call this a B+/A- on reproducibility, better than most of what's out there for this kind of project.

5. Experimental Results

What I ran

All experiments use the same UCI dataset as the article, random_state=42 everywhere a seed is accepted, and an 80/20 stratified train/test split unless noted. On top of reproducing the author's

logistic-regression baseline, I trained three models the article doesn't use (Random Forest, scikit-learn's HistGradientBoostingClassifier as a boosted-tree alternative to XGBoost, and a small MLP as a fair sklearn stand-in for the Keras network), ran 5-fold cross-validation for each, and then repeated the whole comparison on a deduplicated version of the dataset to isolate the leakage effect described in section 2.

Modifications introduced

- Recoded the target as 0/1 (phishing = 1) instead of the original -1/1, purely for readability in plots and metric functions.
- Added Random Forest, HistGradientBoosting and an MLP as comparison models not present in the source article.
- Added 5-fold cross-validation on top of the single train/test split.
- Added a deduplicated-dataset rerun to check for train/test leakage.
- Tried one-hot encoding, a hand-built risk-count feature, and redundant-feature removal (all described in section 3).

Results – raw 80/20 split (matches the article's methodology)

Model	Accuracy	Precision	Recall	F1	MCC	ROC-AUC
Logistic Regression	0.928	0.934	0.900	0.917	0.853	0.978
Random Forest	0.977	0.980	0.967	0.974	0.953	0.996
HistGradientBoosting	0.969	0.971	0.958	0.965	0.937	0.996
MLP (small NN)	0.976	0.978	0.966	0.972	0.951	0.996

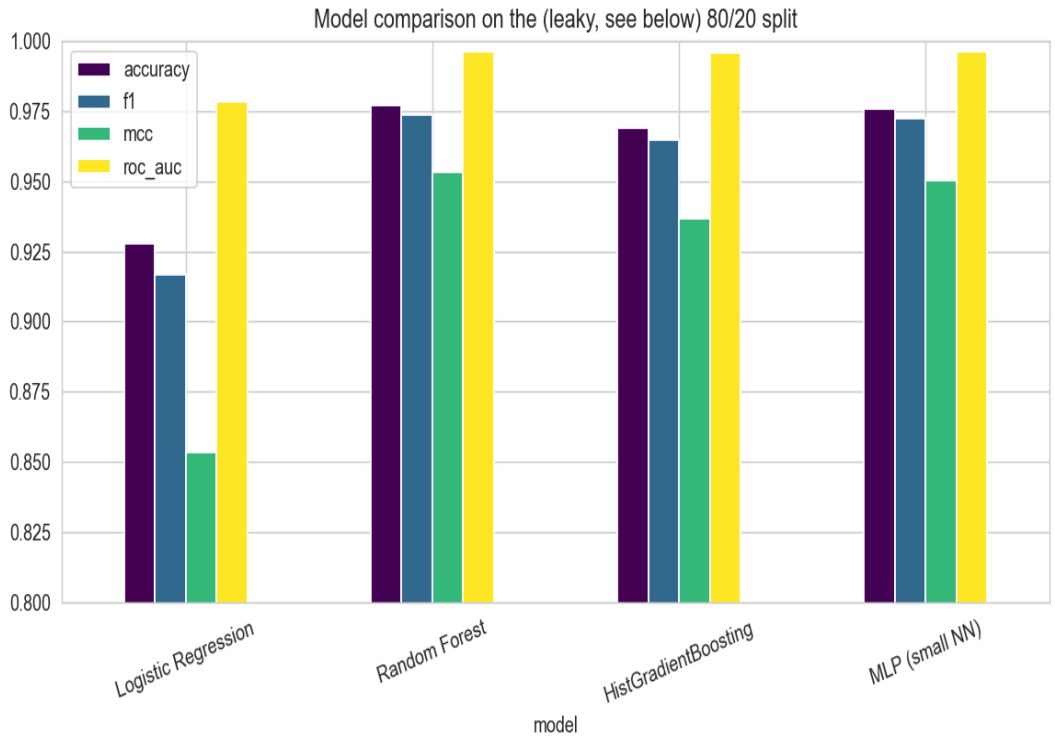

Figure 1. Accuracy / F1 / MCC / ROC-AUC across the four models, raw 80/20 split.

Results – after removing duplicate rows (honest, leak-free estimate)

Model	Accuracy	F1	MCC	ROC-AUC
Logistic Regression	0.934	0.936	0.869	0.983
Random Forest	0.965	0.965	0.929	0.994

Logistic regression is essentially unaffected by deduplication (92.8% → 93.4%, within the kind of variation you'd see across random seeds anyway). Random Forest drops about 1.2 accuracy points (97.7% → 96.5%) once the leaked rows are removed – a real but moderate effect, not the kind of result that should change anyone's mind about whether tree ensembles work well here, but exactly the kind of number that should be reported alongside any "state of the art" claim on this dataset.

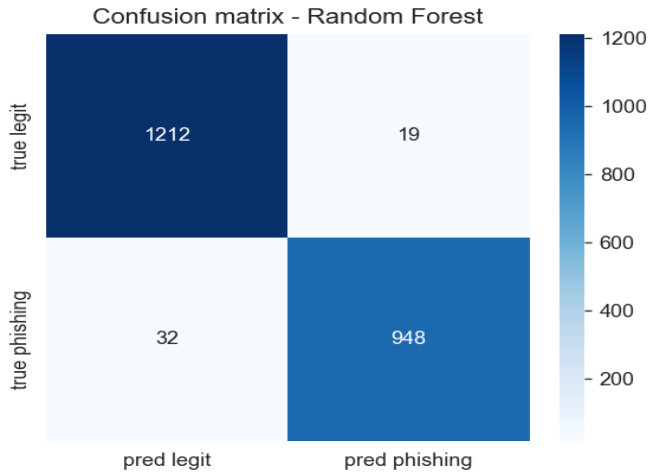

Figure 2. Confusion matrix for the best model (Random Forest) on the raw split.

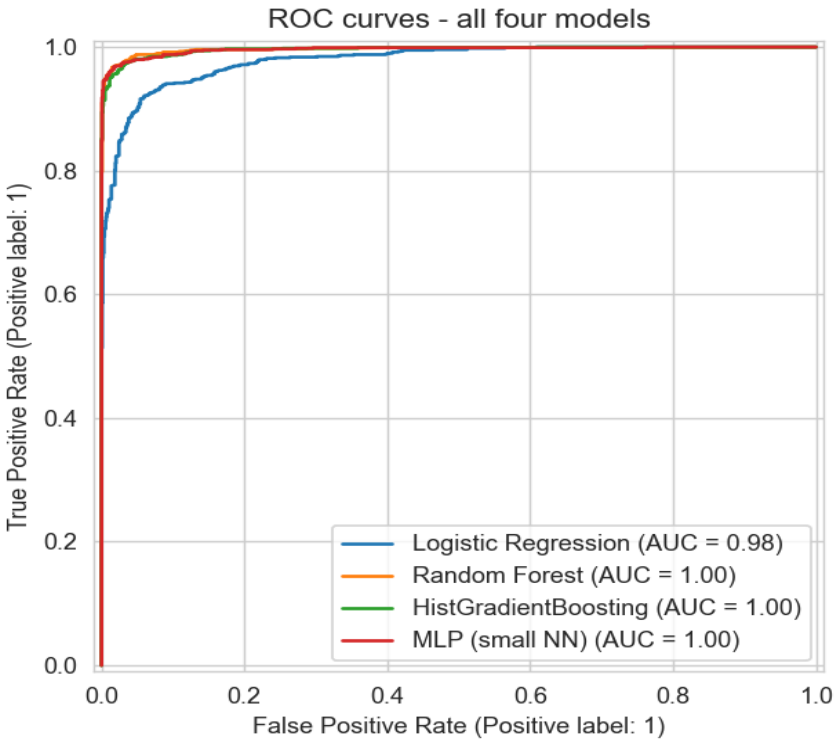

Figure 3. ROC curves for all four models – they're close enough that ROC-AUC alone barely separates them; F1/MCC are more discriminating here.

Encoding / feature-engineering experiment results

Variant	Model	Accuracy	F1
raw ordinal encoding	Logistic Regression	0.928	0.917
one-hot (8 cols)	Logistic Regression	0.939	0.930
raw ordinal encoding	Random Forest	0.977	0.974

one-hot (8 cols)	Random Forest	0.972	0.968
raw + risk_flag_count	Random Forest	0.975	0.972

Feature importance / correlation

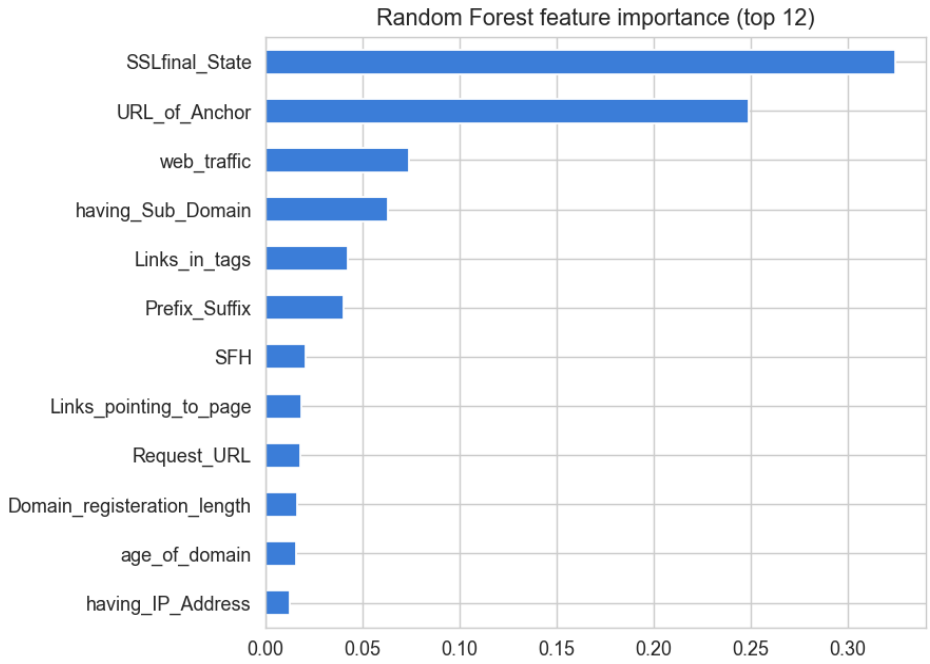

Figure 4. Random Forest feature importance, top 12 – dominated by the same two features flagged by the correlation analysis.

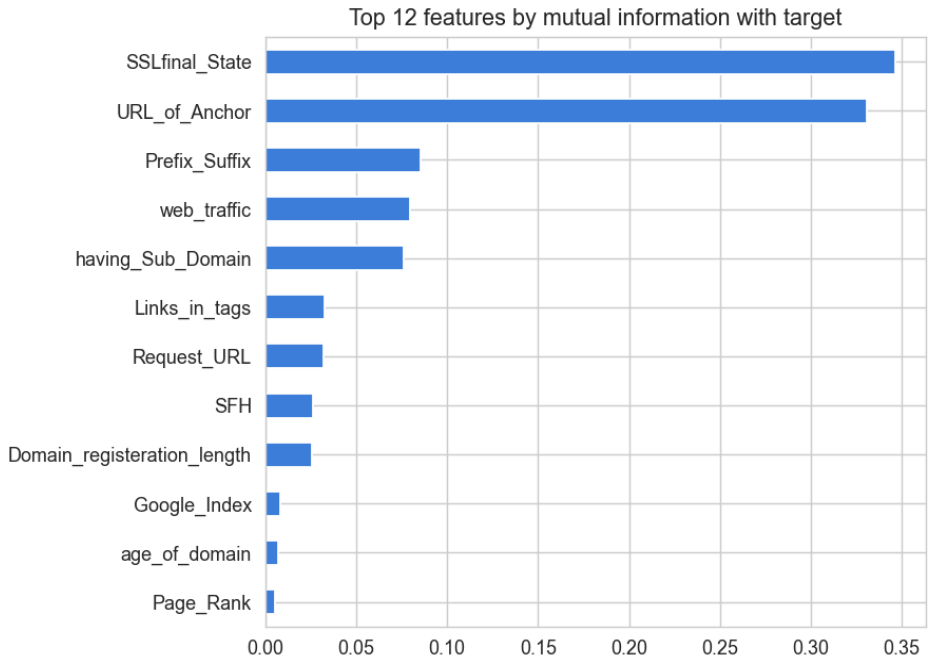

Figure 5. Mutual information with the target, top 12 features.

6. Error Analysis

Looking at the Random Forest's mistakes on the raw test split: 19 false positives (legitimate sites flagged as phishing) and 32 false negatives (phishing sites missed entirely). The pattern in *why* each type of mistake happens is pretty clean and lines up with the feature-importance finding above:

- False positives have a mean SSLfinal_State of about 0.05, near the "suspicious/ambiguous" middle category rather than the "trusted" end. These look like legitimate sites with an imperfect or unusual SSL setup tripping the model's strongest feature.
- False negatives have a mean SSLfinal_State of about 0.75, close to the "trusted" end. These are phishing sites that managed to obtain a valid-looking HTTPS certificate – exactly the scenario that's gotten much easier for attackers since free, automated certificate authorities (Let's Encrypt etc.) became widespread well after this dataset's features were designed in 2014.

Cybersecurity-wise, that second pattern is the one I'd worry about in a real deployment: a feature that was a strong phishing signal a decade ago (a valid SSL cert meant *someone vetted this domain*) has weakened over time as cert issuance got automated and cheap. A model leaning this heavily on one feature is going to degrade as attacker behavior shifts around that exact feature, which is a textbook adversarial/concept-drift risk for security ML, and it's also a direct consequence of this dataset having no temporal information (flagged back in section 1 of the notebook) – there's no way to check whether this drift is already visible *within* the dataset because we don't know when each row was collected.

On the false-positive/false-negative trade-off: this run has more false negatives than false positives, meaning the model currently leans toward under-flagging rather than over-flagging. For a phishing detector specifically I'd lean toward tolerating more false positives in exchange for fewer false negatives – a flagged legitimate site costs a user a moment of friction; a missed phishing site can cost real money or credentials. That argues for tuning the classification threshold down rather than leaving it at the default 0.5, something neither the original article nor my reproduction actually does, but would be the natural next step before any real deployment.

7. Executive Summary

This project reproduces and critiques *"Detecting Phishing Websites using Machine Learning"* by Sayak Paul, a Medium/Intel Software Innovators article that trains logistic regression, a neural network, and a fastai model with entity embeddings on the UCI Phishing Websites dataset (11,055 sites, 30 pre-engineered categorical features), reporting accuracies of 93.71%, 96.52% and 97.02% respectively, with the fastai model's entity embeddings framed as the key driver of the final improvement.

I reproduced the baseline and neural-net results reasonably closely, but couldn't run the fastai model itself due to tooling/version issues, a genuine reproducibility gap rather than a finding against the article. More importantly, I found two things the article doesn't mention: (1) **47% of the dataset's rows are exact duplicates**, and a random train/test split leaks ~65% of the test set back into training as memorized rows, inflating accuracy by roughly 1.2 points for tree-based models once corrected; and (2) a plain, untuned **Random Forest matches or beats the fastai entity-embedding model** even after fixing

that leak (96.5% vs. 97.0%), which undercuts the article's framing of entity embeddings as the thing that gets you to "state of the art" on this particular dataset.

On feature engineering: the 30 features themselves are well designed and grounded in real attacker behavior, but two of them (SSLfinal_State, URL_of_Anchor) account for the large majority of the predictive signal under three independent measures (Spearman correlation, mutual information, and Random Forest importance), which conflicts with the article's stated decision to skip feature selection on the assumption that "every feature contributes." A hand-built aggregate "risk flag count" feature I tried did not improve results, a useful negative result. Error analysis shows the model's mistakes cluster around ambiguous or unexpectedly-valid SSL states, including phishing sites that obtained legitimate-looking certificates, a known weak spot for any detector leaning heavily on certificate validity.

Recommendation: I'd recommend this article and dataset as a solid teaching example for phishing detection with classical ML, but with the caveat that the duplicate-row issue must be checked and corrected before trusting any reported accuracy number, and that the marginal value of the entity-embedding step over a simple Random Forest is, on this evidence, smaller than the article implies.

8. Summing It Up

Problem: binary classification of websites as phishing or legitimate from 30 pre-computed structural/host-based features.

Source: "Detecting Phishing Websites using Machine Learning" by Sayak Paul (Medium / Intel Software Innovators), GitHub repo sayakpaul/Phishing-Websites-Detection.

Dataset: UCI Phishing Websites dataset, 11,055 rows, 30 categorical features, target Result.

Methodology: reproduced the author's logistic-regression baseline; added Random Forest, HistGradientBoosting and an MLP for comparison; ran the comparison again after deduplicating rows; tried one-hot encoding, a custom aggregate feature, and redundant-feature removal; used Spearman/Kendall correlation and mutual information (justified over Pearson given the categorical, non-continuous nature of every feature) to identify which features actually drive the predictions; analyzed false positives and false negatives for the best model.

Main findings of the reproduction: baseline numbers reproduce reasonably closely; the dataset has a serious duplicate-row problem (47% exact duplicates) that inflates naive accuracy estimates by roughly 1.2 points for tree models; a simple Random Forest matches the author's most complex model once that's corrected for; two features dominate the prediction far more than the article's "no feature selection needed" framing suggests.

Were the author's claims supported? Partly. The core claim that ML can detect phishing well on this dataset (90%+ accuracy) is well supported. The claim that the entity-embedding model represents a meaningful, necessary improvement and "matches state of the art" is not well supported by my reproduction – a much simpler model gets you almost all the way there, and the comparison itself wasn't checked for the leakage issue described above.

Most important insight: with a dataset this categorical and this small in feature count, the choice of model matters far less than people might assume, and basic data hygiene (checking for duplicate rows before splitting) matters more than the choice between a neural net and a tree ensemble.

Would I recommend this approach for similar problems? Yes, with caveats: the underlying feature set and the general "structural features + classical ML" approach is a genuinely solid starting point for phishing detection, and I'd reuse it. I would not reuse the entity-embedding step without first establishing that the categorical features in a new dataset actually have enough cardinality/structure to benefit from it, and I would always deduplicate rows before splitting on this specific dataset.

Final conclusion: a well-documented, mostly reproducible tutorial built on a respectable academic dataset, let down a little by an unverified claim about its most sophisticated model and a data-quality issue that nobody using this dataset (the author, me on my first pass, and apparently most of the other tutorials I found while researching this project) seems to have checked for.